

ICPC Notebook Java

Team Reference

Convenciones: Java 17; indices desde cero; intervalos semiabiertos salvo indicacion contraria. Los fragmentos se compilan y prueban desde el repositorio.

Índice

1. Plantilla y entrada **1**
1.1. Main de concurso 1

2. Estructuras de datos **2**
2.1. DSU, Fenwick y segment tree 2

3. Cadenas **3**
3.1. KMP, Manacher, Boyer-Moore y Z 3

4. Matematicas **4**
4.1. Teoria de numeros 4

5. Grafos **6**
5.1. Recorridos y caminos 6

1. Plantilla y entrada

1.1. Main de concurso

Archivo unico listo para un juez en linea.

```java
import java.io.*;
import java.util.*;

/** Standalone template for judges that require a single Main.java file. */
public class Main {
    static final class FastScanner {
        private final InputStream in = new BufferedInputStream(System.in);
        private final byte[] buffer = new byte[1 << 16];
        private int pos, len;

        private int read() throws IOException {
            if (pos >= len) {
                len = in.read(buffer);
                pos = 0;
                if (len < 0) return -1;
            }
            return buffer[pos++] & 0xff;
        }

        String next() throws IOException {
            int c;
            do { c = read(); } while (c >= 0 && c <= ' ');
            if (c < 0) return null;
            StringBuilder token = new StringBuilder();
            while (c > ' ') { token.append((char) c); c = read(); }
```

```
            return token.toString();
        }

        int nextInt() throws IOException { return Integer.parseInt(next()); }
        long nextLong() throws IOException { return Long.parseLong(next()); }
    }

    public static void main(String[] args) throws Exception {
        FastScanner in = new FastScanner();
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        // Solve the problem here. Read with in.nextInt(), write with out.println().
        out.flush();
    }
}
```

2. Estructuras de datos

2.1. DSU, Fenwick y segment tree

Indices desde cero e intervalos semiabiertos.

```
package notebook;

/** Zero-indexed contest data structures. */
public final class DataStructures {
    private DataStructures() {}

    /** Disjoint set union with path compression and union by size. */
    public static final class DSU {
        private final int[] parent, size;
        private int components;
        public DSU(int n) {
            if (n < 0) throw new IllegalArgumentException();
            parent = new int[n]; size = new int[n]; components = n;
            for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        }
        public int find(int x) {
            if (parent[x] != x) parent[x] = find(parent[x]);
            return parent[x];
        }
        public boolean union(int a, int b) {
            a = find(a); b = find(b);
            if (a == b) return false;
            if (size[a] < size[b]) { int t = a; a = b; b = t; }
            parent[b] = a; size[a] += size[b]; components--;
            return true;
        }
        public int size(int x) { return size[find(x)]; }
        public int components() { return components; }
    }

    /** Fenwick tree for point additions and half-open range sums. O(log n) per operation. */
    public static final class Fenwick {
        private final long[] tree;
        public Fenwick(int n) {
            if (n < 0) throw new IllegalArgumentException();
            tree = new long[n + 1];
        }
        public void add(int index, long delta) {
            if (index < 0 || index >= tree.length - 1) throw new IndexOutOfBoundsException();
            for (int i = index + 1; i < tree.length; i += i & -i) tree[i] += delta;
        }
        public long prefixSum(int endExclusive) {
            if (endExclusive < 0 || endExclusive >= tree.length) throw new IndexOutOfBoundsException
();
            long result = 0;
            for (int i = endExclusive; i > 0; i -= i & -i) result += tree[i];
            return result;
        }
        public long rangeSum(int left, int rightExclusive) {
            if (left > rightExclusive) throw new IllegalArgumentException();
            return prefixSum(rightExclusive) - prefixSum(left);
        }
    }

    /** Iterative segment tree for point assignment and half-open range sums. */
    public static final class SegmentTree {
```

```
        private final long[] tree;
        private final int n, base;
        public SegmentTree(long[] values) {
            n = values.length;
            int power = 1;
            while (power < n) power <<= 1;
            base = power;
            tree = new long[2 * base];
            for (int i = 0; i < n; i++) tree[base + i] = values[i];
            for (int i = base - 1; i > 0; i--) tree[i] = tree[2 * i] + tree[2 * i + 1];
        }
        public void set(int index, long value) {
            if (index < 0 || index >= n) throw new IndexOutOfBoundsException();
            int p = base + index;
            tree[p] = value;
            for (p >>= 1; p > 0; p >>= 1) tree[p] = tree[2 * p] + tree[2 * p + 1];
        }
        public long sum(int left, int rightExclusive) {
            if (left < 0 || left > rightExclusive || rightExclusive > n) throw new
IndexOutOfBoundsException();
            long result = 0;
            for (int l = left + base, r = rightExclusive + base; l < r; l >>= 1, r >>= 1) {
                if ((l & 1) != 0) result += tree[l++];
                if ((r & 1) != 0) result += tree[--r];
            }
            return result;
        }
    }
}
```

3. Cadenas

3.1. KMP, Manacher, Boyer-Moore y Z

Opera sobre unidades UTF-16 de Java.

```
package notebook;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** String algorithms operate on Java UTF-16 code units. */
public final class StringsAlgorithms {
    private StringsAlgorithms() {}

    /** KMP match starts, including all n+1 positions for an empty pattern. O(n+m). */
    public static List<Integer> kmp(String text, String pattern) {
        List<Integer> matches = new ArrayList<>();
        if (pattern.isEmpty()) {
            for (int i = 0; i <= text.length(); i++) matches.add(i);
            return matches;
        }
        int[] pi = prefixFunction(pattern);
        for (int i = 0, j = 0; i < text.length(); i++) {
            while (j > 0 && text.charAt(i) != pattern.charAt(j)) j = pi[j - 1];
            if (text.charAt(i) == pattern.charAt(j)) j++;
            if (j == pattern.length()) { matches.add(i - j + 1); j = pi[j - 1]; }
        }
        return matches;
    }

    /** Prefix function. O(n). */
    public static int[] prefixFunction(String s) {
        int[] pi = new int[s.length()];
        for (int i = 1; i < s.length(); i++) {
            int j = pi[i - 1];
            while (j > 0 && s.charAt(i) != s.charAt(j)) j = pi[j - 1];
            if (s.charAt(i) == s.charAt(j)) j++;
            pi[i] = j;
        }
        return pi;
    }

    /** Longest palindromic substring by Manacher, O(n). */
    public static String manacher(String s) {
        if (s.isEmpty()) return "";
```

```java
        int n = s.length(), bestStart = 0, bestLen = 1;
        int[] odd = new int[n], even = new int[n];
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 1 : Math.min(odd[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n && s.charAt(i - k) == s.charAt(i + k)) k++;
            odd[i] = k--;
            int len = 2 * odd[i] - 1;
            if (len > bestLen) { bestLen = len; bestStart = i - odd[i] + 1; }
            if (i + k > r) { l = i - k; r = i + k; }
        }
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 0 : Math.min(even[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n && s.charAt(i - k - 1) == s.charAt(i + k)) k++;
            even[i] = k--;
            int len = 2 * even[i];
            if (len > bestLen) { bestLen = len; bestStart = i - even[i]; }
            if (i + k > r) { l = i - k - 1; r = i + k; }
        }
        return s.substring(bestStart, bestStart + bestLen);
    }

    /** Boyer-Moore bad-character search. O(nm) worst case; UTF-16 aware. */
    public static List<Integer> boyerMoore(String text, String pattern) {
        List<Integer> matches = new ArrayList<>();
        int n = text.length(), m = pattern.length();
        if (m == 0) {
            for (int i = 0; i <= n; i++) matches.add(i);
            return matches;
        }
        int[] last = new int[Character.MAX_VALUE + 1];
        Arrays.fill(last, -1);
        for (int i = 0; i < m; i++) last[pattern.charAt(i)] = i;
        for (int shift = 0; shift <= n - m;) {
            int j = m - 1;
            while (j >= 0 && pattern.charAt(j) == text.charAt(shift + j)) j--;
            if (j < 0) {
                matches.add(shift);
                shift += shift + m < n ? m - last[text.charAt(shift + m)] : 1;
            } else shift += Math.max(1, j - last[text.charAt(shift + j)]);
        }
        return matches;
    }

    /** Z-function: z[i] is longest common prefix of s and s.substring(i). O(n). */
    public static int[] zFunction(String s) {
        int[] z = new int[s.length()];
        for (int i = 1, l = 0, r = 0; i < s.length(); i++) {
            if (i <= r) z[i] = Math.min(r - i + 1, z[i - l]);
            while (i + z[i] < s.length() && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
            if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
        }
        return z;
    }
}
```

4. Matematicas

4.1. Teoria de numeros

Incluye primalidad determinista de 64 bits y CRT generalizado.

```java
package notebook;

import java.math.BigInteger;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/** Integer number theory. Modular operations require mod > 0. */
public final class NumberTheory {
    private NumberTheory() {}

    public static long gcd(long a, long b) {
        if (a == Long.MIN_VALUE || b == Long.MIN_VALUE)
            return BigInteger.valueOf(a).gcd(BigInteger.valueOf(b)).longValueExact();
        a = Math.abs(a); b = Math.abs(b);
```

```
        while (b != 0) { long t = a % b; a = b; b = t; }
        return a;
    }

    public static long lcm(long a, long b) {
        if (a == 0 || b == 0) return 0;
        long product = Math.multiplyExact(a / gcd(a, b), b);
        if (product == Long.MIN_VALUE) throw new ArithmeticException("lcm does not fit in long");
        return Math.abs(product);
    }

    /** Returns [gcd(a,b), x, y] with ax+by=gcd; coefficients must fit in long. */
    public static long[] extendedGcd(long a, long b) {
        BigInteger aa = BigInteger.valueOf(a), bb = BigInteger.valueOf(b);
        BigInteger oldR = aa, r = bb, oldS = BigInteger.ONE, s = BigInteger.ZERO;
        BigInteger oldT = BigInteger.ZERO, t = BigInteger.ONE;
        while (r.signum() != 0) {
            BigInteger q = oldR.divide(r);
            BigInteger nr = oldR.subtract(q.multiply(r)); oldR = r; r = nr;
            BigInteger ns = oldS.subtract(q.multiply(s)); oldS = s; s = ns;
            BigInteger nt = oldT.subtract(q.multiply(t)); oldT = t; t = nt;
        }
        if (oldR.signum() < 0) { oldR = oldR.negate(); oldS = oldS.negate(); oldT = oldT.negate(); }
        return new long[] { oldR.longValueExact(), oldS.longValueExact(), oldT.longValueExact() };
    }

    /** Exact multiplication modulo mod without signed-long overflow. O(log b). */
    public static long modMul(long a, long b, long mod) {
        if (mod <= 0) throw new IllegalArgumentException("mod must be positive");
        long x = Math.floorMod(a, mod), y = Math.floorMod(b, mod), result = 0;
        while (y > 0) {
            if ((y & 1) != 0) result = addMod(result, x, mod);
            y >>>= 1;
            if (y > 0) x = addMod(x, x, mod);
        }
        return result;
    }

    private static long addMod(long a, long b, long mod) {
        return a >= mod - b ? a - (mod - b) : a + b;
    }

    /** a^exponent modulo mod. O(log exponent * log mod). */
    public static long modPow(long a, long exponent, long mod) {
        if (exponent < 0 || mod <= 0) throw new IllegalArgumentException();
        long result = 1 % mod, base = Math.floorMod(a, mod);
        while (exponent > 0) {
            if ((exponent & 1) != 0) result = modMul(result, base, mod);
            exponent >>>= 1;
            if (exponent > 0) base = modMul(base, base, mod);
        }
        return result;
    }

    /** Deterministic Miller-Rabin for signed 64-bit positive integers. */
    public static boolean isPrime(long n) {
        if (n < 2) return false;
        for (long p : new long[] {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
            if (n % p == 0) return n == p;
        }
        long d = n - 1;
        int s = Long.numberOfTrailingZeros(d);
        d >>= s;
        for (long witness : new long[] {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
            long a = witness % n;
            if (a == 0) continue;
            long x = modPow(a, d, n);
            if (x == 1 || x == n - 1) continue;
            boolean passed = false;
            for (int r = 1; r < s; r++) {
                x = modMul(x, x, n);
                if (x == n - 1) { passed = true; break; }
            }
            if (!passed) return false;
        }
        return true;
    }

    /** Primes <= n; O(n log log n) time, O(n) space. */
```

```java
    public static List<Integer> sieve(int n) {
        if (n < 2) return new ArrayList<>();
        boolean[] composite = new boolean[n + 1];
        for (int p = 2; (long) p * p <= n; p++)
            if (!composite[p]) for (int x = p * p; x <= n; x += p) composite[x] = true;
        List<Integer> primes = new ArrayList<>();
        for (int i = 2; i <= n; i++) if (!composite[i]) primes.add(i);
        return primes;
    }

    /** Sorted positive divisors of n; n > 0. O(sqrt n). */
    public static List<Long> divisors(long n) {
        if (n <= 0) throw new IllegalArgumentException();
        List<Long> result = new ArrayList<>();
        for (long d = 1; d <= n / d; d++) {
            if (n % d == 0) {
                result.add(d);
                if (d != n / d) result.add(n / d);
            }
        }
        Collections.sort(result);
        return result;
    }

    /** Exact factorial; n >= 0. */
    public static BigInteger factorial(int n) {
        if (n < 0) throw new IllegalArgumentException();
        BigInteger result = BigInteger.ONE;
        for (int i = 2; i <= n; i++) result = result.multiply(BigInteger.valueOf(i));
        return result;
    }

    /** Generalized CRT. Returns [least nonnegative solution, lcm], or null if inconsistent. */
    public static BigInteger[] chineseRemainder(long[] moduli, long[] residues) {
        if (moduli.length != residues.length) throw new IllegalArgumentException();
        BigInteger x = BigInteger.ZERO, m = BigInteger.ONE;
        for (int i = 0; i < moduli.length; i++) {
            if (moduli[i] <= 0) throw new IllegalArgumentException("moduli must be positive");
            BigInteger next = BigInteger.valueOf(moduli[i]);
            BigInteger target = BigInteger.valueOf(residues[i]).mod(next);
            BigInteger g = m.gcd(next), delta = target.subtract(x);
            if (!delta.mod(g).equals(BigInteger.ZERO)) return null;
            BigInteger reduced = next.divide(g);
            BigInteger k = reduced.equals(BigInteger.ONE) ? BigInteger.ZERO
                : delta.divide(g).multiply(m.divide(g).modInverse(reduced)).mod(reduced);
            x = x.add(m.multiply(k));
            m = m.multiply(reduced);
            x = x.mod(m);
        }
        return new BigInteger[] { x, m };
    }

    /** Newton iteration; derivative must remain nonzero near the root. */
    public static double newtonRaphson(DoubleUnaryOperator f, DoubleUnaryOperator derivative,
                                       double start, double tolerance, int maxIterations) {
        if (!(tolerance > 0) || maxIterations <= 0) throw new IllegalArgumentException();
        double x = start;
        for (int i = 0; i < maxIterations; i++) {
            double fx = f.applyAsDouble(x), dx = derivative.applyAsDouble(x);
            if (Math.abs(fx) <= tolerance) return x;
            if (!Double.isFinite(dx) || dx == 0) throw new ArithmeticException("zero derivative");
            x -= fx / dx;
            if (!Double.isFinite(x)) throw new ArithmeticException("non-finite iterate");
        }
        throw new ArithmeticException("did not converge");
    }
}
```

5. Grafos

5.1. Recorridos y caminos

Grafos dirigidos por defecto; agregar ambas aristas para no dirigidos.

```java
package notebook;

import java.util.ArrayDeque;
```

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/** Zero-indexed graph algorithms. */
public final class Graphs {
    private Graphs() {}

    /** Directed adjacency list. For undirected graphs add both directions. */
    public static List<List<Integer>> newGraph(int n) {
        if (n < 0) throw new IllegalArgumentException();
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        return graph;
    }

    /** Unweighted shortest-path distances; unreachable vertices have -1. O(V+E). */
    public static int[] bfs(List<List<Integer>> graph, int start) {
        int[] dist = new int[graph.size()];
        Arrays.fill(dist, -1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        dist[start] = 0; queue.add(start);
        while (!queue.isEmpty()) {
            int v = queue.remove();
            for (int to : graph.get(v)) if (dist[to] == -1) {
                dist[to] = dist[v] + 1; queue.add(to);
            }
        }
        return dist;
    }

    /** Iterative DFS visitation order. O(V+E). */
    public static List<Integer> dfs(List<List<Integer>> graph, int start) {
        boolean[] seen = new boolean[graph.size()];
        ArrayDeque<Integer> stack = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        stack.push(start);
        while (!stack.isEmpty()) {
            int v = stack.pop();
            if (seen[v]) continue;
            seen[v] = true; order.add(v);
            List<Integer> neighbors = graph.get(v);
            for (int i = neighbors.size() - 1; i >= 0; i--) if (!seen[neighbors.get(i)]) stack.push(
neighbors.get(i));
        }
        return order;
    }

    /** Kahn topological order; throws if the directed graph contains a cycle. */
    public static List<Integer> topologicalSort(List<List<Integer>> graph) {
        int[] indegree = new int[graph.size()];
        for (List<Integer> edges : graph) for (int to : edges) indegree[to]++;
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < indegree.length; i++) if (indegree[i] == 0) queue.add(i);
        List<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
            int v = queue.remove(); result.add(v);
            for (int to : graph.get(v)) if (--indegree[to] == 0) queue.add(to);
        }
        if (result.size() != graph.size()) throw new IllegalArgumentException("graph has a cycle");
        return result;
    }

    public record WeightedEdge(int to, long weight) {}

    /** Dijkstra; requires nonnegative weights. Unreachable vertices have Long.MAX_VALUE. */
    public static long[] dijkstra(List<List<WeightedEdge>> graph, int start) {
        long[] dist = new long[graph.size()];
        Arrays.fill(dist, Long.MAX_VALUE);
        PriorityQueue<long[]> queue = new PriorityQueue<>(Comparator.comparingLong(a -> a[0]));
        dist[start] = 0; queue.add(new long[] {0, start});
        while (!queue.isEmpty()) {
            long[] current = queue.remove();
            long d = current[0]; int v = (int) current[1];
            if (d != dist[v]) continue;
            for (WeightedEdge edge : graph.get(v)) {
                if (edge.weight < 0) throw new IllegalArgumentException("negative edge");
```

```
                if (d > Long.MAX_VALUE - edge.weight) continue;
                long candidate = d + edge.weight;
                if (candidate < dist[edge.to]) {
                    dist[edge.to] = candidate;
                    queue.add(new long[] {candidate, edge.to});
                }
            }
        }
        return dist;
    }

    public record Edge(int from, int to, long weight) {}

    /** Kruskal MST; throws if disconnected. O(E log E). */
    public static long kruskal(int n, List<Edge> edges) {
        List<Edge> sorted = new ArrayList<>(edges);
        sorted.sort(Comparator.comparingLong(Edge::weight));
        DataStructures.DSU dsu = new DataStructures.DSU(n);
        long total = 0;
        for (Edge edge : sorted) if (dsu.union(edge.from, edge.to)) total = Math.addExact(total,
edge.weight);
        if (n > 0 && dsu.components() != 1) throw new IllegalArgumentException("disconnected graph")
;
        return total;
    }
}
```